

Spring Security

Tema 6

Навіщо захищати API?

Зміст лекції

- Архітектурні принципи безпеки та ланцюг фільтрів SecurityFilterChain
- Автентифікація користувачів, AuthenticationManager та алгоритм Argon2id
- Управління сесіями, захист від фіксації та міждоменний обмін CORS
- Безсесійний захист REST API за допомогою токенів JWT та OncePerRequestFilter
- Декларативна авторизація бізнес-логіки через Method Security та SpEL
- Централізована обробка помилок за стандартом RFC 9457 ProblemDetail
- Автоматизоване тестування безпеки та корпоративні стандарти OAuth 2.0

Фундаментальні принципи безпеки програмних систем

- Тріада CIA гарантує конфіденційність, цілісність та доступність даних.
- Ешелонований захист поєднує незалежні контури контролю на рівнях мережі, шлюзу, сервісів і бази даних.
- Принцип найменших привілеїв надає суб'єкту мінімальний набір прав для поточної задачі.
- Підхід Shift-Left переносить автоматизований пошук вразливостей на етапи написання коду та збірки CI/CD.

Парадигма Secure-by-Default у Spring Boot

- Стартер `spring-boot-starter-security` блокує всі HTTP-ендпоінти одразу після підключення.
- За відсутності налаштувань створюється користувач `user` із випадковим згенерованим паролем у консолі.
- Архітектура вимагає точкового дозволу публічних маршрутів замість масового зняття обмежень.
- Повне відключення перевірок є критичним антипатерном навіть на етапі прототипування.

Документація: [Spring Security Getting Started](#)

Розмежування автентифікації та авторизації

- Автентифікація є процесом перевірки справжності суб'єкта та відповідає на запитання про те, хто саме звертається до системи.
- У разі ненадання або недійсності облікових даних сервіс повертає статус HTTP 401 Unauthorized.
- Авторизація полягає у перевірці наявності необхідних прав та дозволів у встановленої особи на виконання конкретної операції.
- При спробі доступу до ресурсу з недостатнім рівнем привілеїв повертається статус HTTP 403 Forbidden.
- Авторизація ніколи не передує автентифікації та спирається на успішно встановлений суб'єкт безпеки.

Перехід від концепцій безпеки до архітектури фільтрів

- Кожен вхідний HTTP-запит має пройти перевірку автентичності та прав ще до потрапляння в контролери Spring MVC.
- Традиційний підхід розміщення перевірок усередині бізнес-методів порушує принцип єдиної відповідальності та створює вразливості.
- Архітектура Spring Security реалізує шаблон перехоплювача на базі стандартних фільтрів сервлетів.
- Далі ми розглянемо, як саме контейнер сервлетів зв'язується з IoC-контейнером Spring через ланцюг фільтрів безпеки.

Архітектура перехоплення запитів у Servlet-контейнері

- Контейнер сервлетів обробляє вхідні HTTP-запити через ланцюг фільтрів сервлетів.
- Клас `DelegatingFilterProxy` реєструється в контейнері сервлетів та виступає містком між сервлетами і контекстом Spring.
- Вся внутрішня логіка делегується компоненту `FilterChainProxy`, що керує бінами ланцюгів безпеки.
- Завдяки такій схемі інфраструктура Spring Security отримує повний доступ до бінів Spring, зокрема сервісів, репозиторіїв та налаштувань.

Документація: [Spring Security Architecture](#)

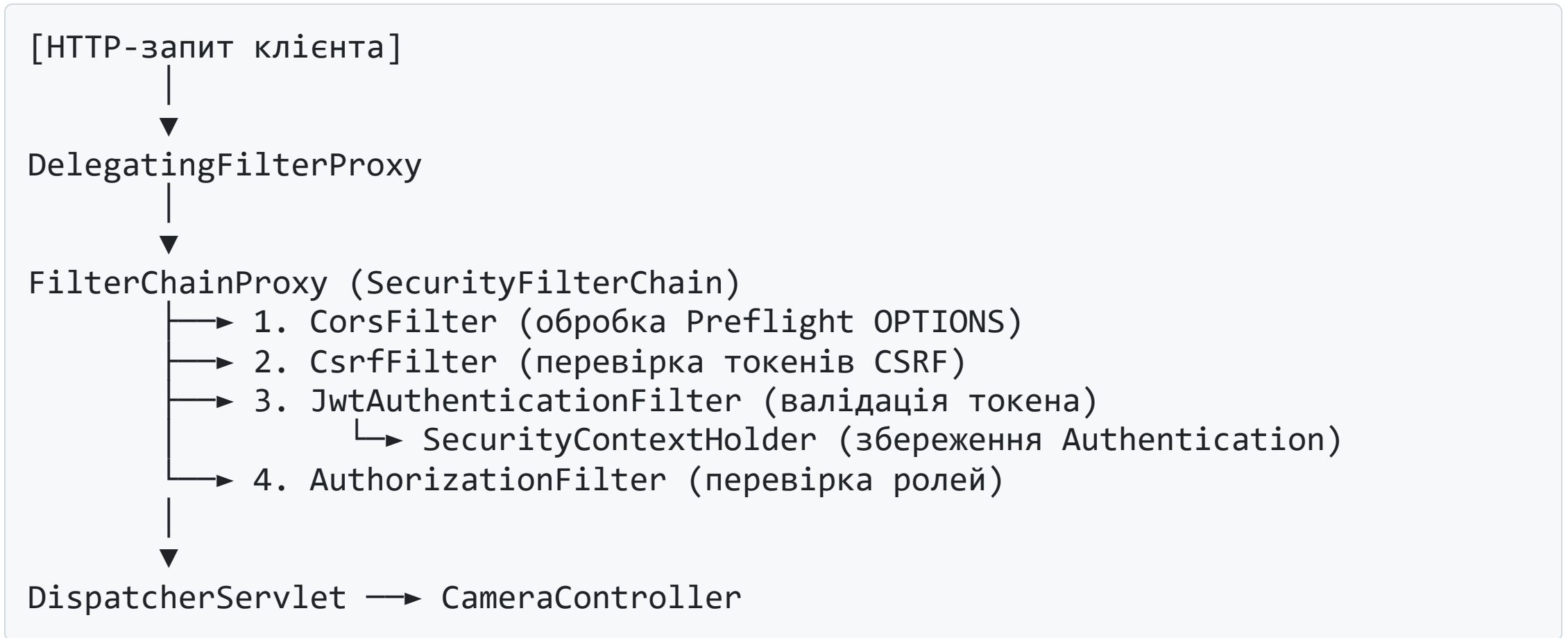
Початкові фільтри ланцюга SecurityFilterChain

- Фільтр `CorsFilter` обробляє попередні запити браузера Preflight OPTIONS та налаштовує заголовки міждоменого обміну.
- Фільтр `CsrfFilter` перевіряє токени захисту від міжсайтової підробки запитів для клієнтів зі збереженням стану.
- Спеціалізовані фільтри автентифікації вилучають облікові дані у вигляді логіна і пароля, tokenів JWT або сесійних кук.

Контекстні та фінальні фільтри ланцюга SecurityFilterChain

- Фільтр `SecurityContextHolderFilter` відновлює або ініціалізує контекст безпеки для поточного запиту.
- Фільтр `ExceptionHandlerFilter` перехоплює винятки безпеки та транслює їх у зрозумілі HTTP-відповіді.
- Фінальний фільтр `AuthorizationFilter` перевіряє відповідність ролей користувача встановленим правилам доступу.

Схема проходження запиту крізь ланцюг SecurityFilterChain



Структура та компоненти інтерфейсу Authentication

- Компонент Principal представляє унікальний ідентифікатор користувача у вигляді об'єкта `UserDetails` , логіна або системного суб'єкта.
- Поле Credentials містить конфіденційні облікові дані, які зазвичай стираються після успішної перевірки автентичності.
- Логічний прапорець Authenticated відображає факт успішного завершення процедури автентифікації.

Модель прав доступу та інтерфейс GrantedAuthority

- Колекція Authorities зберігає незмінювані об'єкти `GrantedAuthority` із переліком наданих ролей та привілеїв.
- Ролі мають конвенційний префікс `ROLE_`, наприклад `ROLE_ADMIN` або `ROLE_OPERATOR`.
- Гранулярні дозволи описують конкретні дії, такі як `CAMERA_READ` чи `CAMERA_DELETE`, без обов'язкових префіксів.
- Такий поділ дозволяє комбінувати високорівневі ролі з гнучкими операційними правами.

Сховище контексту `SecurityContextHolder` та безпека потоків

- Об'єкт `SecurityContextHolder` надає статичний доступ до інтерфейсу `SecurityContext`, що зберігає поточний `Authentication`.
- Історично за замовчуванням використовувалася стратегія збереження контексту у локальній пам'яті потоку через `ThreadLocal`.
- У класичній моделі обробки запиту одним потоком це гарантувало надійну ізоляцію даних між різними викликами.
- Після завершення обробки запиту ланцюг фільтрів зобов'язаний гарантовано очищати контекст за допомогою виклику `clearContext`.

Виклики збереження контексту безпеки у багатопотоковому середовищі

- Архітектура `ThreadLocal` створювалася для систем із фіксованим пулом важких потоків операційної системи.
- З появою віртуальних потоків Project Loom у Java 25 один сервер може одночасно виконувати мільйони легкових задач.
- Застосування традиційного механізму у високонавантажених сервісах призводить до значних перевитрат оперативної пам'яті.
- Далі ми розглянемо, як Java 25 вирішує проблему безпечної передачі контексту за допомогою незмінних `Scoped Values`.

Виклики Java 25 щодо Virtual Threads та обмежень ThreadLocal

- Традиційний `ThreadLocal` створює відчутні накладні витрати оперативної пам'яті при масштабуванні до сотень тисяч віртуальних потоків.
- Небезпека збереження посилань на важкі об'єкти викликає витік пам'яті у спільних пулах задач.
- Віртуальні потоки спроектовані як дешеві короткоживучі сутності, тому змінювані прив'язки суперечать їхній архітектурі.
- У промислових середовищах із високим навантаженням виникає потреба в легковажному та гарантовано незмінному рішенні.

Сучасне рішення Java 25 через механізм Scoped Values

- Клас `ScopedValue` дозволяє передавати незмінні дані вниз за деревом викликів без явного прокидання параметрів у методи.
- Область видимості значення строго обмежена межами блоку виконання коду за допомогою методів `run` або `call`.
- Забезпечує безпечне автоматичне очищення пам'яті відразу після завершення операції.
- Сучасний Spring Security нативно інтегрує Scoped Values для прокидання `SecurityContext` крізь віртуальні потоки.

Документація: [Java 25 Scoped Values JEP 481](#)

Приклад використання Scoped Values для контексту безпеки

```
public final class SecurityScopeRunner {  
  
    public static final ScopedValue<SecurityContext> SECURITY_CONTEXT =  
        ScopedValue.newInstance();  
  
    public static <T> T callWithContext(SecurityContext context, Callable<T> task) throws Exception {  
        return ScopedValue.where(SECURITY_CONTEXT, context)  
            .call(task::call);  
    }  
  
    public static SecurityContext currentContext() {  
        return SECURITY_CONTEXT.orElseGet(SecurityContextHolder::getContext);  
    }  
}
```

Пояснення роботи `Scored Values` у багатопотоковому середовищі

- Статична константа `SECURITY_CONTEXT` створює типізовану комірку незмінюваного зв'язування.
- Метод `ScoredValue.where` фіксує конкретний об'єкт `SecurityContext` виключно для часу виконання переданої задачі.
- Будь-який метод або асинхронний віртуальний потік, породжений усередині блоку, безпечно бачить встановлений контекст.
- Після завершення лямбда-виразу прив'язка миттєво анулюється без ризику залишкового забруднення пам'яті.

Перехід від збереження контексту до ідентифікації користувача

- Контейнер контексту готовий приймати об'єкт автентифікації під час кожного запиту.
- Проте застосунок повинен перевірити справжність користувача та отримати його облікові дані зі сховища.
- Spring Security розмежовує абстракцію перевірки автентичності та механізм завантаження користувачів із бази даних.
- Далі ми розглянемо взаємодію `AuthenticationManager`, `AuthenticationProvider` та інтерфейсу `UserDetails`.

Архітектура автентифікації через менеджери та провайдери

- Інтерфейс `AuthenticationManager` визначає центральний контракт перевірки автентичності через метод `authenticate`.
- Клас `ProviderManager` реалізує стандартну поведінку та містить ланцюжок зареєстрованих компонентів `AuthenticationProvider`.
- Кожен провайдер перевіряє підтримку наданого типу автентифікації за допомогою виклику `supports`.
- Спеціалізовані провайдери підтримують перевірку через базу даних, цифрові підписи токенів або зовнішні каталоги.
- Перший провайдер, який успішно підтверджує облікові дані, повертає повністю заповнений об'єкт `Authentication`.

Контракт сутності користувача та інтерфейс UserDetails

- Метод `getUsername` повертає унікальний логін користувача в системі.
- Метод `getPassword` повертає хешований пароль для перевірки відповідності.
- Метод `getAuthorities` надає перелік наданих ролей та дозволів.
- Спеціальні логічні прапорці контролюють блокування та строк придатності облікового запису і пароля.
- Якщо хоча б один із прапорців повертає від'ємне значення, провайдер автентифікації генерує виняток і блокує вхід.

Приклад оголошення полів сутності користувача

```
@Entity
@Table(name = "users")
public class UserEntity implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserRole role;

    @Column(nullable = false)
    private boolean enabled = true;
```

Приклад реалізації методів інтерфейсу UserDetails

```
@Override  
public Collection<? extends GrantedAuthority> getAuthorities() {  
    return List.of(new SimpleGrantedAuthority(role.name()));  
}
```

```
@Override  
public String getPassword() { return password; }
```

```
@Override  
public String getUsername() { return username; }
```

```
@Override  
public boolean isAccountNonExpired() { return true; }
```

```
@Override  
public boolean isAccountNonLocked() { return true; }
```

```
@Override  
public boolean isCredentialsNonExpired() { return true; }
```

```
@Override  
public boolean isEnabled() { return enabled; }
```

```
}
```

Пояснення реалізації інтерфейсу UserDetails

- Сутність зберігається в реляційній базі даних та відображає обліковий запис оператора або адміністратора системи.
- Метод `getAuthorities` транслює перелічуваний тип ролі в об'єкт `SimpleGrantedAuthority`.
- Значення поля ролі відповідає прийнятому шаблону з префіксом ролей системи.
- Поле пароля містить виключно криптографічний хеш, згенерований безпечним алгоритмом хешування.

Пошук користувачів через контракт `UserDetailsService`

- Контракт містить єдиний метод `loadUserByUsername` , що повертає знайдений об'єкт `UserDetails` .
- Якщо обліковий запис за вказаним логіном не знайдено, сервіс зобов'язаний згенерувати `UsernameNotFoundException` .
- Відокремлює логіку доступу до сховища користувачів від процесу валідації паролів.
- Реалізується як звичайний Spring-бін, що взаємодіє з JPA-репозиторієм або зовнішнім сховищем.

Документація: [Spring Security UserDetailsService](#)

Приклад сервісу завантаження користувача з репозиторію

```
@Service
public class JpaUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public JpaUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        return userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Користувача з логіном " + username + " не знайдено."
            ));
    }
}
```

Пояснення роботи JPA-сервісу користувачів

- Бін впроваджує `UserRepository` через конструктор відповідно до вимог незмінності залежностей.
- Метод `findByUsername` виконує оптимізований запит до таблиці користувачів.
- Застосування `Optional.orElseThrow` гарантує генерацію стандартного винятку `UsernameNotFoundException`.
- Компонент автоматично підхоплюється стандартним провайдером `DaoAuthenticationProvider` під час старту.

Необхідність надійного криптографічного захисту паролів

- Збереження паролів у відкритому вигляді є критичною вразливістю, яка призводить до компрометації всіх облікових записів.
- Зловмисник, отримавши несанкціонований доступ до бази даних, миттєво заволодіває доступами до всіх камер та підсистем.
- Застосування застарілих або занадто швидких функцій хешування не зупиняє сучасні обчислювальні кластери.
- Далі ми розглянемо еволюцію захисту паролів та впровадження сучасного криптографічного стандарту Argon2id.

Еволюція алгоритмів хешування паролів

- Алгоритми MD5 та SHA-1 або SHA-256 створені для швидкого підрахунку контрольних сум і легко ламаються перебором на сучасних відеокартах.
- Адаптивні алгоритми BCrypt та PBKDF2 використовують фактор складності, що штучно уповільнює розрахунок хешу.
- Спеціалізовані чипи ASIC оптимізують паралельне обчислення BCrypt та знижують вартість підбору паролів.
- Сучасним промисловим стандартом визнано алгоритм Argon2id, який переміг у міжнародному конкурсі Password Hashing Competition.

Криптографічний алгоритм Argon2id

- Алгоритм вимагає значного обсягу оперативної пам'яті для розрахунку кожного хешу, що унеможливлює масовий паралельний злом на чипах ASIC.
- Потреба у пам'яті робить злом на графічних процесорах економічно та технічно не вигідним.
- Поєднує захист від атак сторонніми каналами та оптимізацію проти паралельних апаратних зломів.
- Повністю підтримується у Spring Security через офіційний клас `Argon2PasswordEncoder`.

Приклад налаштування кодувальника паролів Argon2id

```
@Configuration(proxyBeanMethods = false)
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new Argon2PasswordEncoder(
            16,    // Довжина солі в байтах
            32,    // Довжина вихідного хешу в байтах
            1,     // Кількість паралельних потоків обчислення
            16384, // Обсяг оперативної пам'яті в кілобайтах (16 МБ)
            3      // Кількість ітерацій обчислення
        );
    }
}
```

Пояснення налаштувань кодувальника Argon2id

- Довжина криптографічної солі в 16 байт гарантує унікальність хешу навіть для однакових вихідних паролів.
- Виділення 16 МБ оперативної пам'яті на один хеш запобігає масовому паралельному перебору паролів злоумисниками.
- Кількість ітерацій забезпечує час хешування на сервері в діапазоні 50-150 мс, що є оптимальним для авторизації.
- Бін `PasswordEncoder` впроваджується в сервіси реєстрації користувачів для кодування паролів перед записом у БД.

Делегований кодувальник DelegatingPasswordEncoder

- Метод `PasswordEncoderFactories.createDelegatingPasswordEncoder` створює гнучкий кодувальник.
- Додає текстовий префікс до згенерованого хешу, наприклад префікс алгоритму в фігурних дужках.
- При перевірці введеного пароля система автоматично обирає відповідний алгоритм за префіксом.
- Дозволяє оновлювати застарілі хеші користувачів на Argon2id безпосередньо у момент їхнього наступного успішного входу.

Збирання компонентів захисту в єдину конфігурацію безпеки

- У нас наявні сутність користувача, репозиторій, сервіс завантаження та стійкий кодувальник паролів.
- Тепер потрібно налаштувати правила доступу до конкретних REST-ендпоінтів та вимкнути непотрібні для API механізми.
- Застарілі підходи наслідування конфігураційних адаптерів повністю замінено функціональним декларативним стилем.
- Далі ми розглянемо побудову біна `SecurityFilterChain` та налаштування міждоменового обміну CORS у Spring Boot 4.1.

Сучасна конфігурація безпеки у Spring Boot 4.1

- Повна відмова від наслідування застарілого класу `WebSecurityConfigurerAdapter`.
- Реєстрація правил здійснюється через фабричний метод біна `SecurityFilterChain`.
- Використання виключно функціонального Lambda DSL для структуризації налаштувань.
- Повна ліквідація застарілих ланцюжків методів, які ускладнювали читання коду конфігурації.
- Застосування статичного методу `Customizer.withDefaults` для ввімкнення стандартних підсистем.

[Документація: Authorize HTTP Requests](#)

Розмежування застосування захисту CSRF для веб-форм та REST API

- Браузер автоматично додає сесійні куки до кожного запиту на цільовий домен, що робить клієнтів зі збереженням стану вразливими до CSRF.
- У безсесійній архітектурі на базі токенів JWT браузер не підставляє Bearer-токен самостійно при переході за стороннім посиланням.
- Вимкнення захисту CSRF є цілком безпечним виключно для stateless API, де авторизація передається через заголовок Authorization.
- Увімкнення CSRF для публічних REST API призводить до помилкового блокування легітимних мобільних та сторонніх клієнтів.

Приклад оголошення класу конфігурації SecurityConfiguration

```
@Configuration(proxyBeanMethods = false)
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfiguration {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http,
        JwtAuthenticationFilter jwtFilter,
        AuthenticationEntryPoint authEntryPoint,
        AccessDeniedHandler accessDeniedHandler) throws Exception {
```

Приклад налаштування базових компонентів безпеки

```
return http
    .csrf(AbstractHttpConfigurer::disable)
    .cors(Customizer.withDefaults())
    .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .exceptionHandling(exceptions -> exceptions
        .authenticationEntryPoint(authEntryPoint)
        .accessDeniedHandler(accessDeniedHandler))
```

Приклад налаштування правил доступу до ендпоінтів у SecurityFilterChain

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/auth/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/v1/cameras/**")
        .hasAnyRole("VIEWER", "OPERATOR", "ADMIN")
    .requestMatchers("/api/v1/cameras/**").hasRole("ADMIN")
    .anyRequest().authenticated())
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
.build();
}
}
```

Пояснення налаштування компонентів SecurityFilterChain

- Відключення захисту CSRF є обґрунтованим, оскільки REST API не використовує браузерні куки для збереження сесій.
- Режим `SessionCreationPolicy.STATELESS` забороняє контейнеру сервлетів виділяти ресурси під серверні `HttpSession`.
- Централізовані обробники `authEntryPoint` та `accessDeniedHandler` перехоплюють помилки доступу на рівні фільтрів.
- Метод `authorizeHttpRequests` задає правила авторизації від найбільш специфічних до загальних правил захисту.
- Фільтр `jwtFilter` монтується перед стандартним фільтром `UsernamePasswordAuthenticationFilter`.

Конфігурація міждоменних запитів CORS

- Клієнтські застосунки, запущені на інших портах, надсилають попередній запит `OPTIONS` у браузері.
- Застосунок повинен явно оголосити дозволені джерела, HTTP-методи та заголовки.
- Конфігурація реєструється у вигляді біна `CorsConfigurationSource` .
- Специфікація вимагає обов'язкового дозволу заголовка `Authorization` для коректної передачі токенів.

Приклад створення параметрів політики CORS

```
@Configuration(proxyBeanMethods = false)
public class CorsConfig {

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:3000", "https://cctv.ukma.edu.ua"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
        config.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Trace-Id"));
        config.setExposedHeaders(List.of("X-Trace-Id"));
        config.setAllowCredentials(false);
        config.setMaxAge(3600L);
    }
}
```

Приклад реєстрації джерела конфігурації CORS для шляхів URL

```
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();  
        source.registerCorsConfiguration("/**", config);  
        return source;  
    }  
}
```

Пояснення налаштування параметрів CORS

- Список дозволених джерел фіксує перелік довірених клієнтських веб-адрес без використання небезпечного шаблону зірочки.
- Дозволені методи охоплюють повний набір REST-операцій разом із методом попередньої перевірки.
- Параметр `setExposedHeaders` відкриває клієнтському JavaScript доступ до читання діагностичного заголовка `X-Trace-Id`.
- Значення часу кешування зберігає результат перевірки в браузері на 1 годину, зменшуючи навантаження на шлюз.

Концепція сесійного стану у сервлетному контейнері

- Об'єкт `HttpSession` забезпечує збереження стану авторизованого користувача на стороні сервера.
- Клієнтський браузер отримує ідентифікатор сесії `JSESSIONID` у заголовок відповіді `Set-Cookie`.
- Наступні запити клієнта передають отриману куку для відновлення безпекового контексту.
- Контейнер сервлетів автоматично відстежує активність сесії та завершує її після таймауту.

Режими створення сесій SessionCreationPolicy

- Політика `ALWAYS` примусово створює сесію у сервлетному контейнері для кожного нового запиту.
- Політика `IF_REQUIRED` створює сесію лише за необхідності збереження контексту автентифікації користувача.
- Політика `NEVER` ніколи не створює нову сесію, але використовує вже створену іншими компонентами.
- Політика `STATELESS` повністю забороняє створення та використання сесій для збереження контексту.

Документація: [Spring Security Session Management](#)

Захист від атаки фіксації сесії Session Fixation

- Атака фіксації полягає у нав'язуванні жертві заздалегідь відомого зловмиснику ідентифікатора сесії.
- Якщо сервер зберігає той самий ідентифікатор після автентифікації, зловмисник отримує доступ до облікового запису.
- Стратегія `changeSessionId` змінює ідентифікатор сесії засобами контейнера сервлетів без зміни атрибутів.
- Стратегія `migrateSession` створює нову сесію та копіює всі наявні дані з попередньої сесії.
- Стратегія `newSession` створює повністю нову чисту сесію без копіювання атрибутів для максимальної ізоляції.

Приклад конфігурації захисту від фіксації та одночасних сесій

```
@Configuration(proxyBeanMethods = false)
public class SessionSecurityConfiguration {

    @Bean
    public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
        return http
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .sessionFixation(SessionFixationConfigurer::migrateSession)
                .maximumSessions(1)
                .maxSessionsPreventsLogin(true)
            )
            .build();
    }
}
```

Приклад реєстрації слухача подій сесій HttpSessionEventPublisher

```
@Bean  
public HttpSessionEventPublisher httpSessionEventPublisher() {  
    return new HttpSessionEventPublisher();  
}  
}
```

Пояснення налаштувань керування сесіями та контролю одночасних входів

- Параметр `maximumSessions(1)` обмежує кількість одночасних активних сесій для одного облікового запису оператора.
- Прапорець `maxSessionsPreventsLogin(true)` забороняє новий вхід, якщо користувач уже пройшов автентифікацію в іншій вкладці або пристрої.
- Якщо встановити значення `false`, новий вхід успішно створить сесію, а попередню сесію буде позначено як завершену.
- Бін `HttpSessionEventPublisher` реєструє слухача подій сервлетного контейнера для своєчасного видалення завершених сесій із реєстру.

Захист сесійних кук JSESSIONID та налаштування завершення сесій

- Сесійна кука повинна мати обов'язковий прапорець `HttpOnly` для захисту від викрадення через сценарії XSS у браузері.
- Прапорець `Secure` гарантує передачу сесійного ідентифікатора виключно зашифрованим каналом HTTPS.
- Атрибут `SameSite=Strict` захищає користувача від атак міжсайтової підробки запитів CSRF.
- Тайм-аут бездіяльності сесії налаштовується у файлі конфігурації для автоматичного завершення покинутих робочих місць.
- Операція виходу з системи Logout зобов'язана виконувати явну інвалідацію `HttpSession` та очищення контексту.

Приклад налаштування параметрів сесій у файлі конфігурації

```
server:  
  servlet:  
    session:  
      timeout: 30m  
      cookie:  
        name: CCTV_SESSION_ID  
        http-only: true  
        secure: true  
        same-site: strict
```

Пояснення налаштування властивостей сесій у YAML

- Властивість `timeout` встановлює ліміт бездіяльності сесії оператора у 30 хвилин.
- Кастомне ім'я куки `CCTV_SESSION_ID` маскує технологічний стек платформи і ускладнює автоматизоване сканування вразливостей.
- Прапорець `same-site: strict` повністю блокує передачу куки при переходах за посиланнями зі сторонніх ресурсів.
- Автоматичне завершення сесії знижує ризик несанкціонованого доступу до панелі керування камерами через залишені браузері.

Розподілені сесії у кластері через Spring Session та Redis

- Збереження сесій у пам'яті окремого вузла створює проблему втрати стану при перезапуску та вимагає неефективних Sticky Sessions.
- Бібліотека `spring-session-data-redis` прозоро заміщує стандартну реалізацію `HttpSession` на розподілене сховище Redis.
- Будь-який вузол кластера миттєво отримує доступ до сесії оператора без прив'язки користувача до конкретного екземпляра сервера.
- Збереження стану в Redis гарантує безперервність роботи користувачів під час планового оновлення та перезапуску вузлів.

Документація: [Spring Session Data Redis](#)

Порівняння сесійної та безсесійної архітектури безпеки

- Сесійний підхід на базі кук є оптимальним вибором для монолітних застосунків із серверним рендерингом сторінок.
- Серверні сесії забезпечують централізоване керування та можливість миттєвого відкликання доступу в адміністративній панелі.
- Безсесійний підхід на базі токенів JWT є стандартом для розподілених мікросервісів, мобільних застосунків та публічних REST API.
- Токени усувають накладні витрати на звернення до централізованої бази сесій під час кожного перегляду відеопотоку.

Перехід від серверних сесій до безсесійної моделі токенів JWT

- Класичні серверні сесії зберігають стан у пам'яті окремого екземпляра сервера, що створює проблеми масштабування.
- При горизонтальному масштабуванні бекенду на десятки контейнерів потрібна дорога синхронізація сесій.
- Безсесійний підхід на базі криптографічних токенів дозволяє кожному мікросервісу валідувати запити автономно.
- Далі ми розглянемо структуру токенів JSON Web Token за стандартом RFC 7519 та їхній повний життєвий цикл.

Архітектура токенів доступу JWT за стандартом RFC 7519

- Самодостатність токена забезпечує передачу ідентифікатора користувача, строку придатності та ролей безпосередньо у його тілі.
- Відсутність сесійного стану на сервері звільняє мікросервіси від потреби звертатися до бази даних під час кожної перевірки прав.
- Токен передається клієнтом у стандартному HTTP-заголовку авторизації з типом Bearer.
- Складається з трьох частин, розділених крапками, що містять заголовок, корисне навантаження та підпис.

Специфікація: [RFC 7519 JSON Web Token](#)

Анатомія структури JWT із заголовком, корисним навантаженням та підписом

- Заголовок токена Header визначає службовий тип токена та алгоритм криптографічного підпису.
- Корисне навантаження Payload містить набір стандартних та користувацьких тверджень про суб'єкт.
- До стандартних полів належать суб'єкт, час видачі та точний момент завершення дії токена.
- Користувацькі твердження зберігають надані ролі, ідентифікатор сесії або організації.
- Цифровий підпис Signature формується хешуванням заголовка та корисного навантаження із секретним ключем сервера.

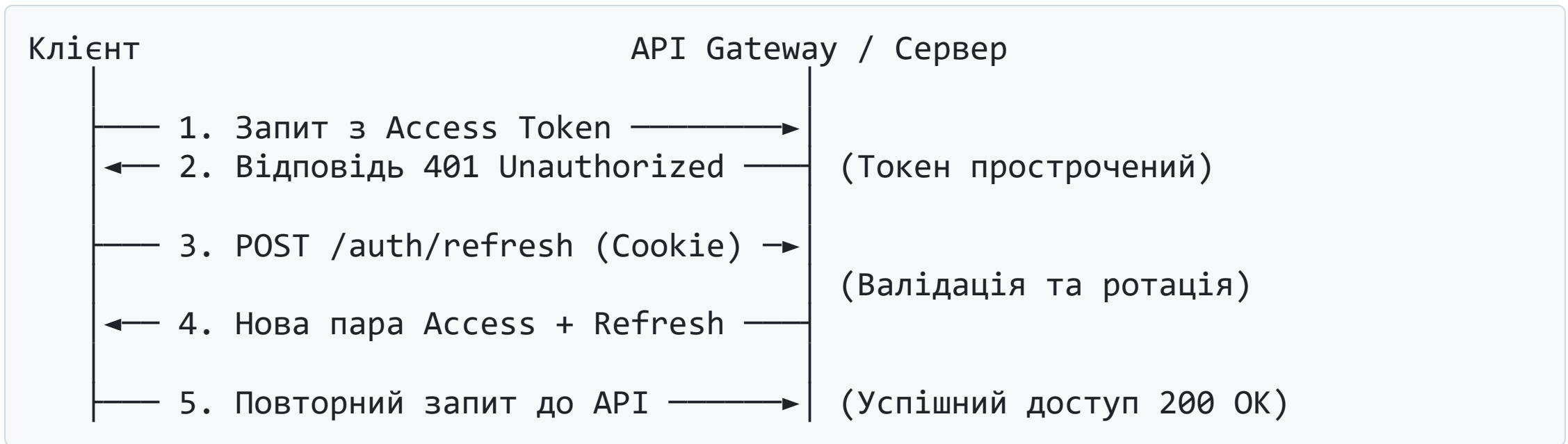
Критичні аспекти безпеки під час роботи з JWT

- Заборонено зберігати конфіденційні персональні дані у відкритому корисному навантаженні, оскільки воно лише кодується у Base64Url.
- Будь-який клієнт або перехоплювач трафіку може декодувати та прочитати корисні дані токена.
- Категорично заборонено зберігати в токені паролі, номери банківських карток чи конфіденційні персональні дані.
- Для збереження конфіденційності потрібен стандарт шифрування JWE, а не звичайний підпис JWS.
- Секретний ключ для підпису повинен мати достатній рівень ентропії для унеможливлення підбору за словником.

Архітектура використання пари tokenів Access та Refresh

- Короткоживучий token доступу Access Token на 10–15 хвилин мінімізує вікно атаки у разі його перехоплення.
- Довгоживучий token оновлення Refresh Token зберігається у захищеній від читання JavaScript куці HttpOnly.
- Механізм ротації Refresh Token генерує новий token оновлення під час кожного звернення, інвалідуючи попередній.
- Виявлення спроби повторного використання старого токена розцінюється як крадіжка та миттєво блокує весь ланцюг сесій.

Схема оновлення токена доступу за допомогою Refresh Token



Управління життєвим циклом tokenів Access та Refresh

- Короткоживучий token доступу Access Token має рекомендований час дії від 5 до 15 хвилин.
- Довгоживучий token для оновлення сесії Refresh Token діє від кількох днів до тижнів.
- Клієнт пред'являє Refresh Token виключно на спеціалізований ендпоінт оновлення для отримання нової пари tokenів.
- Безпечне збереження Refresh Token на фронтенді здійснюється в захищених куках для запобігання атакам.
- Такий підхід захищає token від викрадення через сценарії атак XSS на стороні браузера.

Приклад записів для роботи з автентифікацією

```
public record LoginRequest(  
    @NotBlank String username,  
    @NotBlank String password  
) {}  
  
public record AuthResponse(  
    String accessToken,  
    String tokenType,  
    long expiresIn  
) {  
    public static AuthResponse of(String token, long expiresIn) {  
        return new AuthResponse(token, "Bearer", expiresIn);  
    }  
}
```

Пояснення моделей запиту та відповіді автентифікації

- Використання Java Records забезпечує компактність та незмінність переданих даних.
- Анотація `@NotBlank` гарантує валідацію наявності логіна та пароля перед викликом сервісу.
- Відповідь `AuthResponse` явно повертає тип токена `Bearer` відповідно до стандарту OAuth 2.0.
- Поле `expiresIn` повідомляє клієнту точний час валідності токена в секундах для планування ротації.

Приклад ініціалізації сервісу генерації tokenів JWT

```
@Service
public class JwtService {

    private final SecretKey signingKey;
    private final long expirationMillis;

    public JwtService(
        @Value("${security.jwt.secret}") String secret,
        @Value("${security.jwt.expiration-minutes:15}") long expirationMinutes) {
        this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMillis = expirationMinutes * 60 * 1000;
    }
}
```

Приклад методу генерації токена JWT

```
public String generateToken(UserDetails userDetails) {  
    Instant now = Instant.now();  
    List<String> roles = userDetails.getAuthorities().stream()  
        .map(GrantedAuthority::getAuthority)  
        .toList();  
  
    return Jwts.builder()  
        .subject(userDetails.getUsername())  
        .claim("roles", roles)  
        .issuedAt(Date.from(now))  
        .expiration(Date.from(now.plusMillis(expirationMillis)))  
        .signWith(signingKey)  
        .compact();  
}
```

Пояснення генерації токенів у JwtService

- Секретний ключ зчитується з зовнішньої конфігурації та перетворюється в безпечний об'єкт `SecretKey`.
- Час створення та закінчення дії обчислюються на базі поточного моменту через `Instant.now`.
- Список ролей витягується з колекції `GrantedAuthority` та записується в користувацьке твердження ролей.
- Метод `signWith` виконує криптографічний підпис згенерованого набору тверджень безпечним алгоритмом.

Вимоги стандарту OWASP до секретних ключів підпису JWT

- Алгоритми сімейства HMAC-SHA вимагають криптографічної довжини ключа не менше 256 біт, тобто мінімум 32 випадкових байти.
- Бібліотека JJWT автоматично викидає `WeakKeyException` під час запуску, якщо переданий ключ має недостатню довжину.
- Категорично заборонено зберігати секретні ключі у відкритому вигляді у файлах конфігурації репозиторію сирцевого коду.
- Ключі мають передаватися виключно через захищені змінні оточення або централізовані сховища секретів.
- Для розподілених мікросервісів рекомендовано застосовувати асиметричні алгоритми підпису RS256 або ES256.

Приклад парсингу та вилучення тверджень токена JWT

```
@Service
public class JwtValidatorService {

    private final SecretKey signingKey;

    public JwtValidatorService(@Value("${security.jwt.secret}") String secret) {
        this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(signingKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}
```

Приклад валідації користувача та строку дії токена JWT

```
public String extractUsername(String token) {  
    return extractAllClaims(token).getSubject();  
}  
  
public boolean isTokenValid(String token, UserDetails userDetails) {  
    String username = extractUsername(token);  
    Date expiration = extractAllClaims(token).getExpiration();  
    return username.equals(userDetails.getUsername()) && expiration.after(new Date());  
}
```

Пояснення криптографічної перевірки токена

- Метод `verifyWith` ініціалізує синтаксичний аналізатор перевіркою коректності цифрового підпису.
- Якщо токен підроблено або змінено хоча б один байт корисного навантаження, генерується виняток підпису.
- Якщо поточний час перевищує термін придатності, парсер генерує виняток простроченого токена.
- Метод `isTokenValid` переконується у відповідності суб'єкта токена поточному користувачу та актуальності строку дії.

Приклад оголошення компонентів фільтра OncePerRequestFilter

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtValidatorService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtValidatorService jwtService,
                                   UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }
}
```

Приклад вилучення токена Bearer з заголовка запиту

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                FilterChain filterChain) throws ServletException, IOException {

    String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        filterChain.doFilter(request, response);
        return;
    }
    String token = authHeader.substring(7);
```

Приклад перевірки валідності токена та завантаження деталей

```
try {  
    String username = jwtService.extractUsername(token);  
    if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {  
        UserDetails userDetails = userService.loadUserByUsername(username);  
        if (jwtService.isTokenValid(token, userDetails)) {
```

Приклад створення автентифікації та збереження в контексті

```
var authToken = new UsernamePasswordAuthenticationToken(  
    userDetails, null, userDetails.getAuthorities());  
authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));  
SecurityContextHolder.getContext().setAuthentication(authToken);  
}  
}
```

Приклад надійної обробки винятків у фільтрі JWT

```
    } catch (JwtException | IllegalArgumentException | AuthenticationException ex) {  
        SecurityContextHolder.clearContext();  
        authenticationEntryPoint.commence(request, response,  
            new BadCredentialsException("Невалідний або прострочений токен JWT", ex));  
        return;  
    }  
  
    filterChain.doFilter(request, response);  
}  
}
```

Пояснення надійної обробки помилок автентифікації у фільтрі

- Клас наслідує `OncePerRequestFilter`, що гарантує одноразове виконання логіки фільтра для одного HTTP-запиту.
- Перевіряється наявність заголовка `Authorization` та префікса `Bearer`; у разі їх відсутності запит передається далі.
- З токена вилучається логін; перевірка запобігає повторній автентифікації, якщо користувач уже є в контексті.
- При виникненні `JwtException` метод `commence` точки входу негайно повертає 401 Unauthorized за стандартом RFC 9457.
- Завершення виконання через оператор `return` унеможлиблює потрапляння невалідного запиту до бізнес-контролерів.

Приклад структури та налаштування фільтра автентифікації

```
public class JwtLoginFilter extends UsernamePasswordAuthenticationFilter {  
  
    private final JwtService jwtService;  
    private final ObjectMapper objectMapper;  
    private final AuthenticationEntryPoint authenticationEntryPoint;  
  
    public JwtLoginFilter(AuthenticationManager authenticationManager,  
                          JwtService jwtService,  
                          ObjectMapper objectMapper,  
                          AuthenticationEntryPoint authenticationEntryPoint) {  
        super(authenticationManager);  
        this.jwtService = jwtService;  
        this.objectMapper = objectMapper;  
        this.authenticationEntryPoint = authenticationEntryPoint;  
        setRequiresAuthenticationRequestMatcher(  
            new AntPathRequestMatcher("/api/v1/auth/login", "POST"));  
    }  
}
```

Приклад перевірки облікових даних та видачі токена у фільтрі

```
@Override
public Authentication attemptAuthentication(HttpServletRequest request, HttpServletResponse response) {
    try {
        LoginRequest loginRequest = objectMapper.readValue(request.getInputStream(), LoginRequest.class);
        var authRequest = UsernamePasswordAuthenticationToken.unauthenticated(
            loginRequest.username(), loginRequest.password());
        setDetails(request, authRequest);
        return getAuthenticationManager().authenticate(authRequest);
    } catch (IOException ex) {
        throw new AuthenticationServiceException("Помилка читання запиту автентифікації", ex);
    }
}

@Override
protected void successfulAuthentication(HttpServletRequest request, HttpServletResponse response,
                                         FilterChain chain, Authentication authResult) throws IOException {
    UserDetails userDetails = (UserDetails) authResult.getPrincipal();
    String token = jwtService.generateToken(userDetails);
    long expiresInSeconds = jwtService.getExpirationMillis() / 1000;
    response.setStatus(HttpServletResponse.SC_OK);
    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
    objectMapper.writeValue(response.getWriter(), AuthResponse.of(token, expiresInSeconds));
}
}
```

Пояснення механізму видачі токенів через ланцюг фільтрів

- Фільтр перехоплює вхідний запит на адресу входу та вилучає облікові дані з потоку JSON.
- Компонент `AuthenticationManager` делегує перевірку пароля зареєстрованому провайдеру автентифікації.
- Метод успішної автентифікації генерує підписаний токен JWT та записує відповідь у вихідний потік.
- Видача токена у фільтрі ізолює автентифікацію від прикладних контролерів і зменшує поверхню атаки.

Перехід від шлюзової безпеки до захисту бізнес-логіки

- Фільтри безпеки перевіряють автентичність запиту на вході в контролер, але не контролюють внутрішні виклики.
- Бізнес-методи можуть викликатися з черг повідомлень, внутрішніх планувальників завдань або суміжних сервісів.
- Принцип ешелонованого захисту вимагає перевірки бізнес-правил безпосередньо над методами доменного шару.
- Далі ми розглянемо декларативний захист бізнес-методів за допомогою анотацій Method Security та виразів SpEL.

Декларативний захист бізнес-логіки через Method Security

- Застосування анотації `@EnableMethodSecurity` на конфігураційному класі активує захист на рівні Spring AOP.
- Дозволяє перевіряти права доступу безпосередньо над методами бізнес-сервісів або репозиторіїв.
- Запобігає несанкціонованому виклику бізнес-операцій з внутрішніх сервісів, черг повідомлень чи планувальників.
- Повністю замінила застарілі конфігурації глобальної безпеки попередніх поколінь.

Документація: [Method Security in Spring Security](#)

Анотації Method Security та вирази SpEL

- Анотація `@PreAuthorize` виконує перевірку правил доступу перед викликом цільового методу.
- Анотація `@PostAuthorize` перевіряє права після завершення методу та дозволяє аналізувати повернутий результат.
- Анотації `@PreFilter` та `@PostFilter` автоматично фільтрують елементи вхідних або вихідних колекцій на базі прав користувача.
- Вираз `hasRole` вимагає наявності права з конвенційним префіксом ролей системи.
- Вираз `hasAuthority` виконує точну перевірку рядка дозволу без додавання службових префіксів.

Приклад перевірки ролей на рівні методів сервісу

```
@Service
public class CameraManagementService {

    private final CameraRepository cameraRepository;

    public CameraManagementService(CameraRepository cameraRepository) {
        this.cameraRepository = cameraRepository;
    }

    @PreAuthorize("hasAnyRole('VIEWER', 'OPERATOR', 'ADMIN')")
    public List<CameraResponse> getAllCameras() {
        return cameraRepository.findAll().stream()
            .map(CameraResponse::fromEntity)
            .toList();
    }

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteCamera(Long cameraId) {
        cameraRepository.deleteById(cameraId);
    }
}
```

Приклад перевірки аргументів методу за допомогою SpEL

```
@PreAuthorize("hasRole('OPERATOR') and #command.bitrate() <= 5000")  
public void updateStreamQuality(Long cameraId, UpdateStreamCommand command) {  
    // Оновлення параметрів відеопотоку оператором у межах ліміту бітрейту  
}  
}
```

Пояснення авторизації викликів бізнес-методів через SpEL

- Метод `getAllCameras` дозволено виконувати будь-якому автентифікованому користувачу системи з базовою роллю.
- Метод `deleteCamera` є критичною адміністративною дією і захищений суворою перевіркою адміністративної ролі.
- Метод `updateStreamQuality` демонструє доступ до значень аргументів виклику за допомогою синтаксису параметрів.
- Якщо оператор спробує перевищити встановлений ліміт бітрейту, перехоплювач згенерує помилку до входу в метод.

Забезпечення прозорості та стандартизації помилок доступу

- Несанкціоновані запити або виклики з простроченими токенами повинні отримувати зрозумілу діагностичну відповідь.
- Винятки безпеки виникають на рівні фільтрів або AOP-проксі, тому стандартні контролери їх не перехоплюють.
- Повернення порожніх відповідей зі статусами 401 або 403 ускладнює налагодження клієнтських фронтенд-застосунків.
- Далі ми розглянемо обробку винятків безпеки на базі офіційного стандарту RFC 9457 ProblemDetail.

Централізована обробка винятків безпеки

- Фільтри сервлетів виконуються до того, як запит досягне центрального диспетчера та контролерів.
- Компонент `AuthenticationEntryPoint` перехоплює помилки відсутності або недійсності облікових даних та повертає статус 401 Unauthorized.
- Компонент `AccessDeniedHandler` обробляє випадки недостатніх привілеїв у автентифікованого користувача та повертає статус 403 Forbidden.
- Для збереження узгодженості формату помилок у Spring Boot 4.1 використовується стандарт RFC 9457 `ProblemDetail`.

Приклад оголошення класу точки входу AuthenticationEntryPoint

```
@Component
public class ProblemDetailAuthenticationEntryPoint implements AuthenticationEntryPoint {

    private final ObjectMapper objectMapper;

    public ProblemDetailAuthenticationEntryPoint(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException) throws IOException {
```

Приклад заповнення властивостей структури ProblemDetail

```
ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(  
    HttpStatus.UNAUTHORIZED,  
    "Для доступу до ресурсу потрібна валідна автентифікація."  
);  
problemDetail.setTitle("Помилка автентифікації");  
problemDetail.setType(URI.create("https://cctv.ukma.edu.ua/errors/unauthorized"));  
problemDetail.setProperty("timestamp", Instant.now());  
problemDetail.setProperty("path", request.getRequestURI());
```

Приклад відправки стандартизованої HTTP-відповіді про помилку

```
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);  
        response.setContentType(MediaType.APPLICATION_PROBLEM_JSON_VALUE);  
        response.setCharacterEncoding(StandardCharsets.UTF_8.name());  
        objectMapper.writeValue(response.getWriter(), problemDetail);  
    }  
}
```

Пояснення роботи обробника помилок автентифікації

- Клас реалізує інтерфейс `AuthenticationEntryPoint` та монтується в ланцюг конфігурації обробки помилок.
- Фабричний метод `ProblemDetail.forStatusAndDetail` конструює об'єкт помилки за офіційним стандартом RFC 9457.
- Встановлюється правильний заголовок типу вмісту для стандартизованого формату проблемного JSON.
- Всі помилки безпеки повертаються клієнтам у єдиному машиночитному форматі без витоку внутрішніх трасувань стека.

Перехід до автоматизованого тестування підсистеми безпеки

- Конфігурація безпеки є критичним компонентом, де навіть одна помилка в шаблоні шляху може відкрити конфіденційні дані.
- Ручна перевірка через Postman або curl є трудомісткою і не захищає від випадкових регресій при зміні коду.
- Повноцінні наскрізні тести із запуском реальної бази даних та генерацією живих JWT-токенів працюють занадто повільно.
- Далі ми розглянемо легковажне тестування шару безпеки за допомогою `@WebMvcTest`, `MockMvc` та анотацій `@WithMockUser`.

Інфраструктура автоматизованого тестування безпеки

- Анотація `@WebMvcTest` завантажує мінімальний веб-контекст разом із фільтрами Spring Security.
- Анотація `@WithMockUser` симулює запит від імені автентифікованого користувача із заданими ролями без створення токенів.
- Анотація `@WithAnonymousUser` явно перевіряє поведінку системи для неавторизованих відвідувачів.
- Тести перевіряють як коректність бізнес-відповідей, так і надійність повернення статусів 401 та 403.

[Документація: Testing Spring Security](#)

Приклад конфігурації MockMvc-тесту

```
@WebMvcTest(CameraController.class)
@Import(SecurityConfiguration.class)
class CameraControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private CameraManagementService cameraService;

    @MockitoBean
    private JwtValidatorService jwtService;

    @MockitoBean
    private UserDetailsService userDetailsService;

    @Test
    void getCameras_WhenUnauthenticated_ShouldReturnUnauthorized() throws Exception {
        mockMvc.perform(get("/api/v1/cameras"))
            .andExpect(status().isUnauthorized());
    }
}
```

Приклад тестування доступу з мокованим користувачем WithMockUser

```
@Test
@WithMockUser(username = "operator", roles = {"OPERATOR"})
void deleteCamera_WhenForbiddenRole_ShouldReturnForbidden() throws Exception {
    mockMvc.perform(delete("/api/v1/cameras/42"))
            .andExpect(status().isForbidden());
}

@Test
@WithMockUser(username = "admin", roles = {"ADMIN"})
void deleteCamera_WhenAdminRole_ShouldReturnNoContent() throws Exception {
    mockMvc.perform(delete("/api/v1/cameras/42"))
            .andExpect(status().isNoContent());
}
}
```

Пояснення результатів перевірки безпеки в тестах

- Перший тест верифікує, що анонімний запит без облікових даних відхиляється статусом HTTP 401 Unauthorized.
- Другий тест перевіряє рольову матрицю: оператор не має прав видаляти камери, тому отримує HTTP 403 Forbidden.
- Третій тест підтверджує успішне виконання адміністративної дії користувачем із роллю `ADMIN`.
- Тести виконуються за мілісекунди без запуску реального сервера та без звернення до бази даних.

Перспективи розвитку корпоративної безпеки та нативна збірка

- Локальна автентифікація логіном і паролем є базовим рішенням, але великі підприємства вимагають єдиної точки входу (SSO).
- Делегування доступу стороннім інтеграціям регламентується протоколом OAuth 2.0 та специфікацією PKCE.
- Новітній напрям кібербезпеки передбачає повну відмову від паролів на користь криптографії пристроїв Passkeys.
- Для хмарних розгортань критичним стає час холодного старту артефакту під компілятор GraalVM AOT.

Корпоративна авторизація та призначення протоколу OAuth 2.0

- Протокол OAuth 2.0 регламентує делеговану авторизацію сторонніх застосунків без передачі пароля користувача.
- Надбудова OpenID Connect забезпечує ідентифікацію користувача за допомогою токена ID Token.
- Використання стандарту позбавляє клієнтські застосунки необхідності обробляти та зберігати облікові дані.

Ролі та учасники взаємодії у протоколі OAuth 2.0

- Власник ресурсів Resource Owner представляє кінцевого користувача або власника захищеного облікового запису.
- Клієнт Client представляє клієнтський застосунок або сервіс, що запитує доступ до захищених ресурсів.
- Сервер авторизації Authorization Server перевіряє особу користувача та видає токени доступу.
- Сервер ресурсів Resource Server надає доступ до бізнес-API за наявності валідного токена доступу.

Захист публічних клієнтів за стандартом PKCE

- Специфікація PKCE є обов'язковим стандартом для безпечної авторизації відкритих клієнтів.
- Клієнт генерує одноразовий випадковий рядок та створює його криптографічний хеш перед початком входу.
- Запит на авторизацію передає лише відкритий хеш, а фінальний запит на отримання токена надає оригінальний секрет.
- Запобігає перехопленню коду авторизації шкідливими програмами або розширеннями браузера.
- Spring Security повністю підтримує стандарт PKCE для клієнтських конфігурацій OAuth 2.0.

Безпарольна автентифікація через WebAuthn та Passkeys

- Стандарт FIDO2 та WebAuthn базується на асиметричній криптографії відкритого та закритого ключів.
- Технологія Passkeys генерує та надійно зберігає закритий криптографічний ключ безпосередньо на пристрої користувача.
- Сервер зберігає виключно відкритий ключ і надсилає криптографічний виклик, який пристрій підписує біометрією.
- Повністю усуває загрозу фішингу, оскільки цифровий підпис прив'язаний до точного доменного імені сайту.
- Нові версії Spring Security підтримують нативну інтеграцію з протоколами WebAuthn.

Оптимізація Spring Security для GraalVM Native Image

- У нативному образі GraalVM заборонено створення нових класів та динамічних проксі під час роботи застосунку.
- Spring Boot 4.1 генерує статичні підказки на етапі збірки артефакту для всіх захищених сервісів.
- Ланцюг фільтрів `SecurityFilterChain` формується детерміновано без потреби в динамічній реєстрації.
- Забезпечує мілісекундний холодний запуск захищених мікросервісів у безсерверних середовищах.

Типові помилки конфігурації та архітектури безпеки

- Вимкнення захисту CSRF у веб-застосунках із сесійними куками, що відкриває можливість для атак міжсайтової підробки запитів.
- Збереження паролів у відкритому вигляді або використання застарілих та скомпрометованих алгоритмів хешування MD5 і SHA-1.
- Плутанина між методами `hasRole` та `hasAuthority`, що призводить до відмов доступу через дублювання префікса `ROLE_ROLE_`.
- Відсутність очищення контексту через `SecurityContextHolder.clearContext()` при перемиканні потоків або завершенні обробки.

Типові помилки роботи з токенами доступу та авторизацією

- Використання занадто коротких або статично прописаних у коді секретних рядків для підпису токенів HMAC.
- Збереження конфіденційної інформації (паролів, персональних даних) у відкритому корисному навантаженні Payload токена.
- Відсутність перевірки строку придатності токена перед виконанням важких запитів до бази даних користувачів.
- Покладання на приховування кнопок та посилань у графічному інтерфейсі клієнта замість серверного захисту методів.

Висновки

- Підсистема Spring Security у Spring Boot 4.1 реалізує надійний захист за принципом Secure-by-Default.
- Модель ланцюга фільтрів `SecurityFilterChain` на базі Lambda DSL забезпечує прозору та лаконічну конфігурацію.
- Використання Scored Values у Java 25 усуває проблеми масштабування пам'яті у багатопотокових віртуальних середовищах.
- Алгоритм Argon2id гарантує максимальну криптографічну стійкість збереження облікових даних до сучасних апаратних атак.
- Архітектура безсесійних JWT у поєднанні з декларативним Method Security та MockMvc забезпечує комплексний ешелонований захист API.

Індивідуальне завдання

1. Підключити стартер `spring-boot-starter-security` та створити простий тестовий ендпоінт для перевірки доступу.
2. Сконфігурувати `SecurityFilterChain` для сесійної автентифікації з формою входу `formLogin` або `httpBasic`.
3. Налаштувати тестового користувача в пам'яті через `InMemoryUserDetailsManager` або властивості `application.properties`.
4. Задати назву сесійної куки через властивість `server.servlet.session.cookie.name` та перевірити налаштування стандартного фільтра `logout`.

Індивідуальне завдання

1. Відкрити захищений ендпоінт у браузері або HTTP-клієнті та зафіксувати перенаправлення на сторінку входу.
2. Успішно пройти автентифікацію з тестовими даними та перевірити наявність куки сесії у заголовках відповіді сервера.
3. Надіслати наступний запит із передачею отриманої куки та переконатися у збереженні стану без повторного введення пароля.
4. Виконати запит виходу на адресу `/logout` і перевірити анулювання сесії та видалення сесійної куки.
5. Завантажити архів проєкту на DistEdu.

Групове завдання

1. Створити кількох тестових користувачів із різними ролями (наприклад, через файл початкових даних або при старті програми).
2. Налаштувати бін `PasswordEncoder` та вивести в логи приклад згенерованого хешу пароля.
3. Реалізувати клас `UserDetailsService` для вибірки облікових записів через JPA-репозиторій із генерацією винятку `UsernameNotFoundException` .

Групове завдання

1. Сконфігурувати безсесійну (Stateless) безпеку через `SessionCreationPolicy.STATELESS` у `SecurityFilterChain` на відміну від сесійного індивідуального завдання.
2. Налаштувати підтримку CORS для веб-клієнта та обробку винятків через RFC 9457 `ProblemDetail`.
3. Реалізувати сервіс токенів доступу (JWT) та зареєструвати фільтри автентифікації й валідації із захистом через `AuthenticationEntryPoint`.

Групове завдання

1. Надіслати запит без токена на захищений ендпоінт і переконатися в отриманні статусу 401 Unauthorized.
2. Виконати запит входу з обліковими даними користувача та отримати згенерований токен доступу.
3. Повторити запит до захищеного ресурсу із заголовком `Authorization: Bearer <token>` і отримати статус 200 OK.
4. Перевірити доступ користувача з недостатніми правами та переконатися в отриманні статусу 403 Forbidden.
5. Завантажити архів проєкту та посилання на репозиторій на DistEdu.